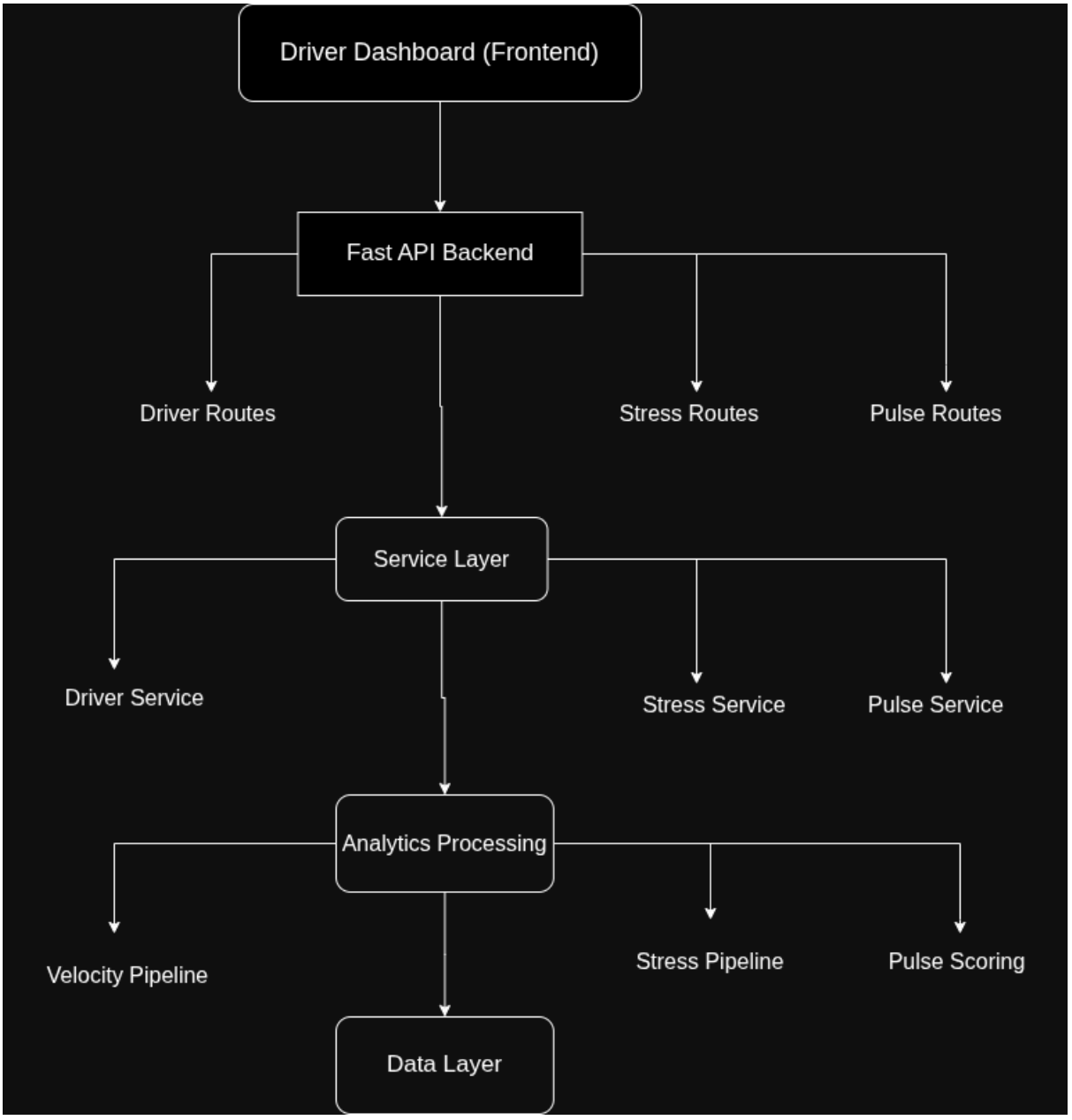
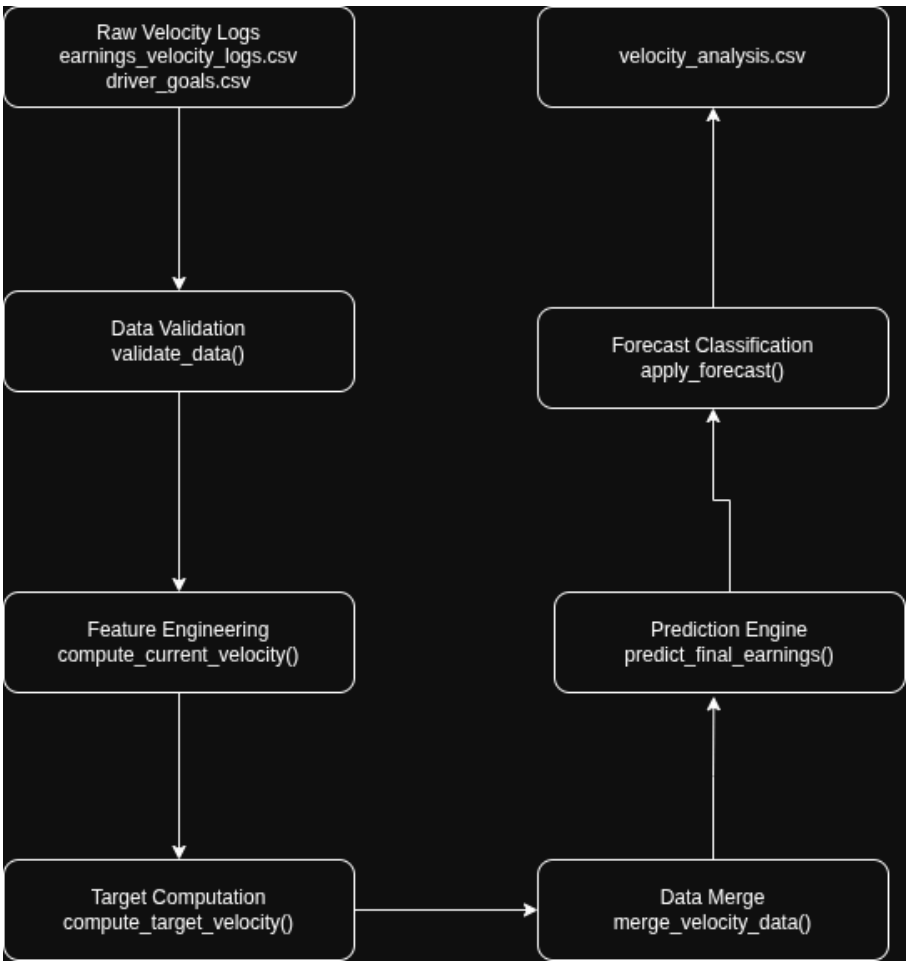
ARCHITECTURE DIAGRAM

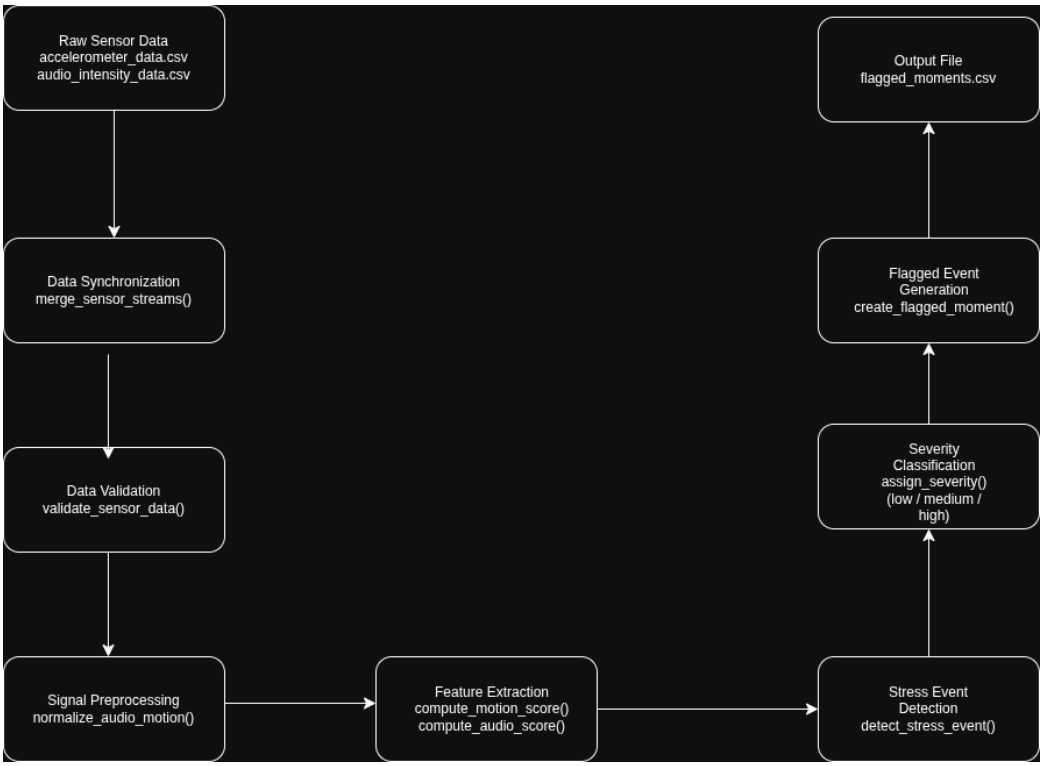
1)Overall System Architechture



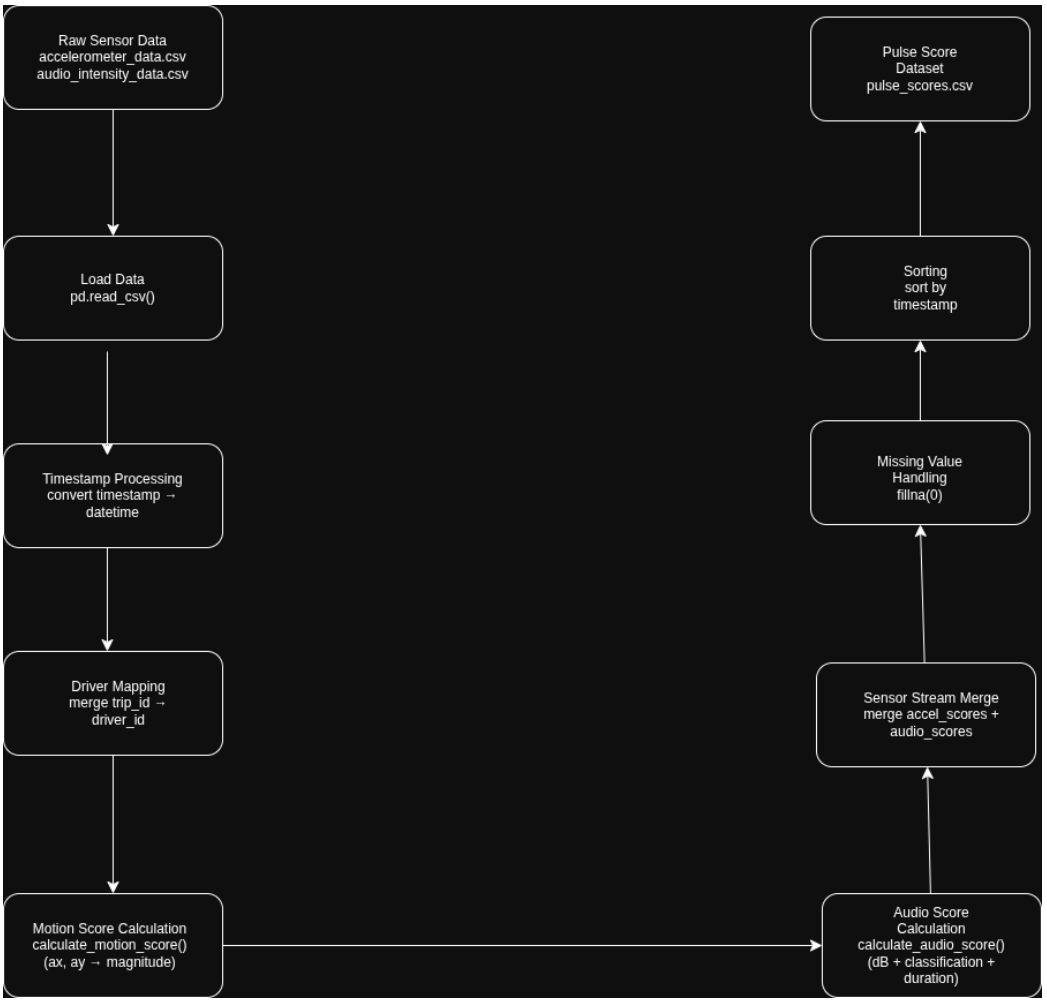
2) Velocity Pipeline



3)Stress detection pipeline



4)Pulse Scores



Why is our system design scalable and reliable?

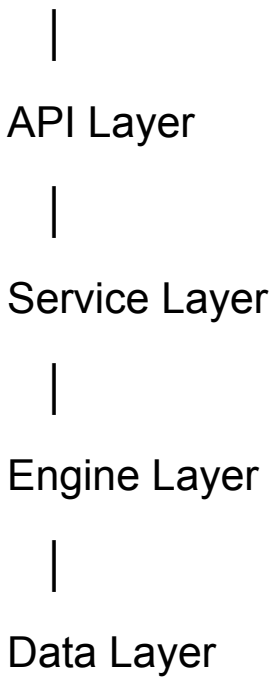
We designed our system using a **layered architecture** that separates responsibilities across the **API layer, service layer, analytics/engine layer, and data layer**. This separation allows each component to operate independently, which improves both **scalability** and **reliability** of the system.

A.Layered Architechture

1.Overall:-

We structured the system into four primary layers:

Frontend



Each layer has a clearly defined responsibility, which reduces coupling between components and makes the system easier to maintain and scale.

2.API Layer

The **API layer** is implemented using FastAPI and is responsible for handling HTTP requests from the frontend dashboard.

The API layer only manages request handling and response formatting, while the actual computation is delegated to lower layers. This keeps the API lightweight and allows it to scale horizontally if request volume increases.

3.Service Layer

The **service layer** orchestrates the main workflows of the system. For example, the velocity computation pipeline is executed through the service function:

```
run_velocity_pipeline();
```

This layer coordinates multiple processing steps including:

- data loading
- validation
- velocity computation
- goal merging
- prediction generation
- forecast classification

  run_stress_detection();

This process includes:

- analyzing motion and audio patterns
- detecting stress-triggering conditions
- classifying event severity (low, medium, high)
- generating flagged stress events for trips

  generate_pulse_scores();

This process involves:

- loading accelerometer and audio sensor data
- mapping trip information to drivers
- computing motion scores from acceleration signals
- computing audio scores based on sound intensity and classification
- merging sensor streams by timestamp

By separating orchestration logic into a service layer, we ensure that the API layer remains simple while complex processing is managed in a dedicated layer.

4.Engine Layer

The engine layer contains the core analytical logic of the system. This includes functions responsible for calculating

driver performance metrics such as validate_data(),compute_target_velocity(),etc.

In addition to velocity analytics, this layer also contains the analysis engines for pulse scoring and stress detection. These components process raw sensor data streams and generate behavioral insights about the driver.

The pulse scoring engine computes motion and audio scores from accelerometer and audio intensity data using functions such as calculate_motion_score() and calculate_audio_score() These scores are then merged and organized into a continuous pulse timeline

Similarly, the stress detection engine analyzes these behavioral signals to identify abnormal driving conditions or stressful in-vehicle environments. It evaluates motion and audio patterns and flags events with different severity levels using the stress detection pipeline

This layer focuses purely on business logic and computation. By isolating the analytical engine from the API and service layers, we make it easier to modify or extend the analytics without affecting other components of the system

5.Data Layer

The data layer manages the storage and retrieval of system data. In our current implementation, this layer uses CSV files to store:

- driver profiles
- driver goals
- trip data
- velocity logs

- Accelerometer sensor data
- Audio intensity data
- processed analytics outputs

The pipeline processes raw data and generates a consolidated output file (`velocity_analysis.csv,pulse_scores.csv,flagged_moments.csv`) that is used by the API layer to serve dashboard responses efficiently.However usage of csv files is only for the purpose of Hackathon. In real world systems, proper database systems will be used.

2.Scalability of the System

First, the API layer is **stateless**, meaning each request is processed independently without relying on server-side session state. This allows multiple instances of the API to run simultaneously behind a load balancer if system traffic increases.

Second, the velocity computation pipeline supports **incremental recomputation**. When a driver updates their goal, we recompute the velocity analysis only for that specific driver rather than recalculating the entire dataset. Similarly, the **pulse scoring and stress detection pipelines process sensor data independently for each trip or driver**, allowing computations to be performed selectively rather than recomputing all historical data.This significantly reduces computational overhead and allows the system to handle larger numbers of drivers efficiently.

Third, the use of a **pipeline-based processing architecture** enables future parallelization.Individual driver computations for velocity estimation, pulse scoring, and stress detection could be distributed across multiple workers or processing nodes if the dataset grows significantly. Since each driver's data can be processed independently, the system naturally supports parallel processing.

Finally, precomputing analytics and storing them in the generated output file ensures that API requests do not trigger heavy computations, which reduces response latency and improves overall throughput.

3. Reliability

Our system is also designed to be reliable.

We include a **data validation stage** in the engine layer to detect issues such as negative earnings or invalid elapsed hours. This prevents corrupted input data from propagating through the analytics pipeline.Similar validation checks are applied when processing accelerometer and audio sensor data used for pulse scoring and stress detection.

We also implement **safe numerical handling** to avoid runtime errors such as division by zero or infinite values. For example, we replace zero denominators with small constants and handle infinite or missing values appropriately.

In addition, the system **handles missing data gracefully**. If velocity history, goal information, or sensor readings are unavailable for a driver, the API returns safe fallback values rather than failing. For example, missing motion or audio

readings are treated as neutral scores, allowing the pulse scoring pipeline to continue processing without interruption.

Another reliability feature is the **pipeline execution during server startup**, which ensures that the analytics outputs are always up to date when the API begins serving requests.

Finally, the modular architecture isolates failures between components. For instance, if the stress detection module encounters an issue, the velocity computation and dashboard APIs continue functioning independently.Similarly, pulse scoring, velocity analytics, and stress detection are implemented as separate pipelines, ensuring that failures in one analytical component do not disrupt the entire system.